

RPC框架

🔍 | ⓘ 正文 1.1 万字 (12.9 页) | 浏览 28 次 (1 人) 查看全部 >

🔗 AI 速览

RPC介绍一

什么是RPC?

rpc：远程调用过程，官方定义：：可以像调用本地的函数一样调用远程函数的一类通信协议的统称。

那么这里什么是本地函数，什么又是远程函数呢？需要结合微服务架构来理解。

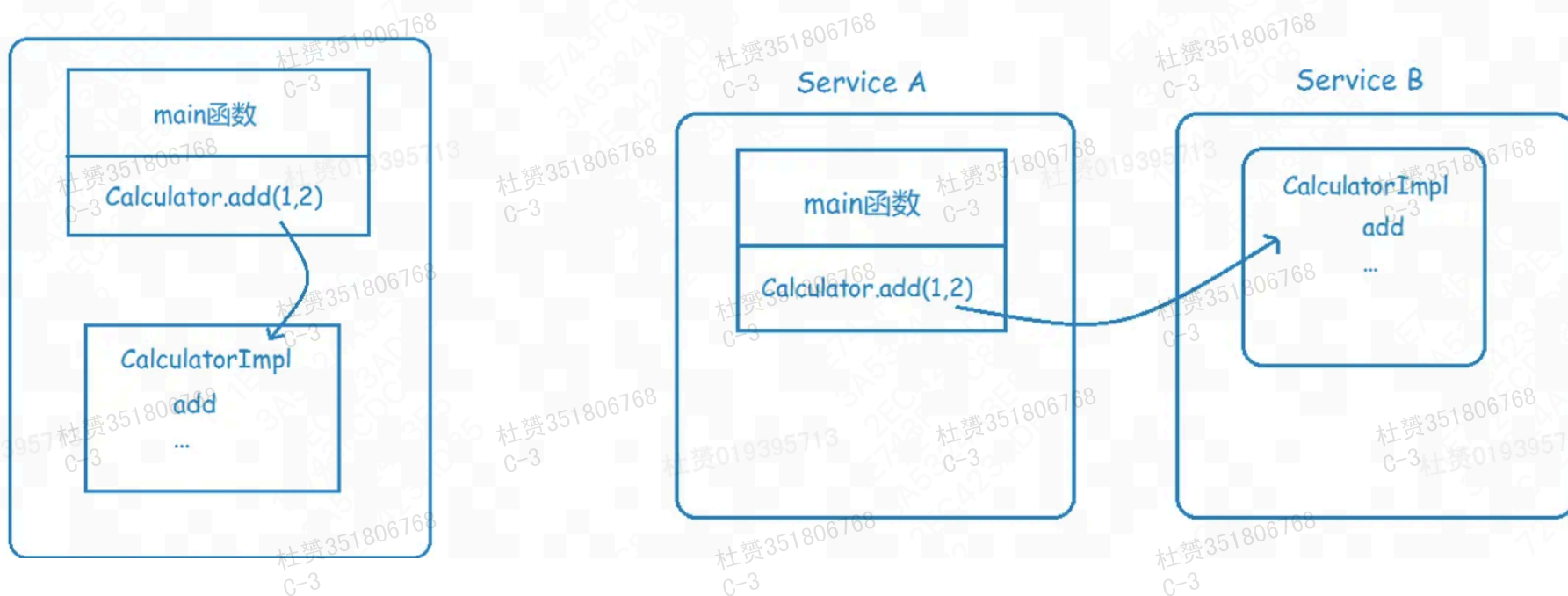
在我们以前的单体应用中，所有的业务逻辑与方法都包含在同一个服务里，在这个服务中，我们定义接口、实现其并再调用其它方法，最终在一个服务中实现所有的业务逻辑。

但是随着服务体量和逻辑复杂性的增长，我们需要将代码，从功能模块的角度进行拆分，同一模块功能的代码拆分到一个服务中，单独迭代部署，这就拆分出了一个微服务。

那么，在微服务场景下，不同服务间要怎样才能实现接口的互相调用，就是RPC框架解决的核心问题。

RPC提供了一种简洁的方式，使我们不用考虑服务间底层的网络通信等逻辑，通过它代理就能完成服务间的远程调用，我们只需要专注于真正的业务接口及代码逻辑编写即可。

单体应用 与 微服务架构 下的函数调用示意图（核心点在：调用方与服务方不在一个内存空间里，通常位于两台不同的服务器上）：



RPC和HTTP区别

当看到RPC框架的最终目的是实现两个服务间的互相调用时，有人可能会问，http协议不就可以实现这样的服务间通信吗，为什么我们还需要rpc框架呢？

也有一些人认为将这两个名词放在一起比较就是一个愚蠢的问题，因为框架与协议就不是一个维度上的事情，更加无从比起，所以接下来，我们先来看一下为什么它们有时会被放在一起提及：

共同点：都位于应用层，可以处理传输层tcp协议粘包、拆包等内容

RPC框架的核心能力有：

- (1) 简化我们编写服务间调用时的代码，使我们不需要关心底层socket网络通信、序列化等逻辑，就可以快速的实现远程服务间的接口调用。
- (2) RPC框架可以选择更底层的网络传输协议（如TCP）、和更高效的二进制序列化传输协议（如protoBuf、thrift）来进行网络通信和序列化，提升通信效率并减小性能消耗。
- (3) RPC框架提供了丰富的服务发现、监控、负载均衡、服务治理能力，有效支持了分布式场景下服务间调用的各种问题。

由于RPC框架和HTTP协议诸多的不同特性，使得其通常被应用在不同的场景，即RPC框架通常被应用在分布式系统中的服务间调用，HTTP协议通常被应用在Web应用中的访问请求。

不同点：

1、RPC框架是一种架构思想，http是真正的应用层通信协议

- RPC：并不是真正的通信协议，其是微服务架构场景下，能够实现远程过程调用的框架的统称，核心是一种框架思想，而非具体框架或协议。（Thrift、Dubbo等才是真正的RPC框架实现）
- HTTP：是真正的应用层通信协议，遵守协议规范请求就可以实现一次端到端调用。

2、传输协议、效率可能不同

真正的端到端通信是RPC框架中的一个组成部分，RPC框架中这一部分的实现可以基于tcp、udp、http 或一些其它协议，当基于tcp协议通信时，RPC框架的通信效率将天然高于http协议。

- RPC：可以基于TCP、UDP、HTTP 或其它协议。当RPC框架底层使用TCP协议，再在上层自己封装时，可以让请求报文体积更小，另外使用HTTP2.0协议，也可以很好地减少报文体积，提高传输效率。
- HTTP：基于HTTP协议。如果是HTTP1.1的协议，请求中会包含很多无用的内容；如果是基于HTTP2.0，那么性能还是比HTTP1.1有较大提高的。

3、传输的内容和性能消耗不同

- RPC：可以采用thrift、protobuf (gRPC)等更高效的二进制传输协议，以二进制的格式序列化并传输编解码后的数据，性能消耗更小。（所以更适合分布式系统中的服务间调用）
- HTTP：传输的数据大多为文本格式，如HTML、JSON等，请求体通过json来序列化数据，字节大小和序列化耗时都比二进制方式要更消耗性能。（基于请求-响应模式的文本协议，主要用于Web应用等）

4、服务发现、监控、负载均衡、服务治理能力不同

- RPC：通常有注册中心作为中心服务保存服务信息（如：zookeeper等），基本都自带了负载均衡策略，有服务治理能力，能做到下游服务新增、重启、下线时自动通知，不影响上游调用方。

服务间调用时会复用线程池中连接，且通常维护tcp长连接。

- HTTP：根据DNS解析域名得到ip地址，负载均衡需要配置Nginx，HAProxy实现，需要事先通知，当下游服务新增、重启、下线时，需要事先通知上游，修改Nginx/HAProxy配置。

5、代码编写方式不同

- RPC：根本能力是可以像调用本地的函数一样调用远程函数，调用的入参、返回的数据都会被框架自动序列化或定义接口时定义的类的对象，不需要像http协议一样关注请求头等内容。

● HTTP：调用的请求需要满足http协议的规则。

RPC怎么做到的

简述调用流程

RPC框架下的一次调用，可以通俗解释为：当客户端想要调用某个其它服务提供的RPC接口时，先设置被调用的服务名称，接着像本地调用一样构造入参、编写接口调用代码，

RPC框架会通过动态代理构建客户端调用类的代理对象，然后调用到同样已被动态代理，且已启动并注册到注册中心的服务端接口实现类对象那里，

注册中心是RPC框架管理所有可以被调用服务的枢纽，提供了服务发现能力，维护了每个服务所有可调用ip地址的列表，客户端通过注册中心得到被调用服务的机器ip地址列表，

再根据本地的负载均衡策略确定最终调用的机器的ip地址，接着序列化请求入参、依赖RPC底层的网络传输能力，发送网络请求到目标服务。

服务端根据调用请求头中的接口名、方法名信息，确定本次请求想要调用的方法，接着对接收到的入参反序列化，复用服务调用线程池传入入参，使用被动态代理的服务端对象执行接口逻辑，

接口逻辑执行完毕后，将结果经服务端序列化、网络传输，再传输给客户端服务，客户端服务对返回数据反序列化，至此得到最终结果，这样就完成了一次RPC框架下的服务间接口调用。

RPC框架抽象

需要说明的是：在上述过程中，我们只需要关注最开始的，设置被调用服务名称、构造调用入参、编写接口调用代码这几步操作，后面的所有逻辑都由RPC框架来帮我们完成，

在服务端也是类似，我们只需要定义并标注期望由RPC框架管理的接口，然后就可以专心的编写接口里的业务逻辑了，

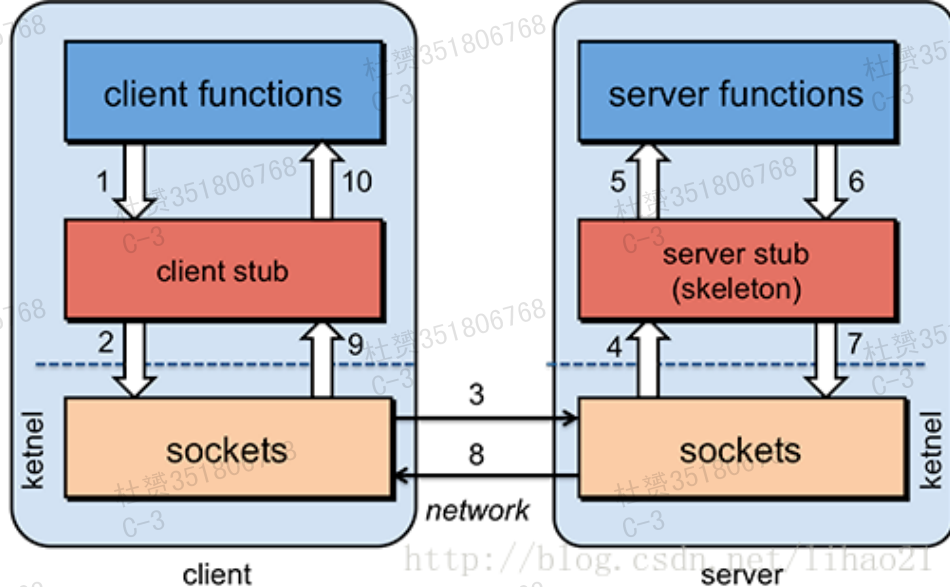
最终我们在客户端和服务端得到的调用参数和调用结果，就如同本地调用函数的参数和返回值一样，是函数定义中的对象类型。

RPC框架将服务间调用中的各种复杂逻辑及难以解决的问题，分别划分模块，寻求最优解，一一解决，使其对我们完全变成黑盒逻辑。

RPC调用的必要元素为：（1）客户端：指定目标调用服务（在美团是每个服务唯一的appkey）、构造调用入参、编写接口调用代码。（2）服务端：被调用接口定义及实现。

将上述的RPC框架执行步骤抽象成实际代码中的对象模型，即如下图所示：

其中，我们所说的RPC框架动态代理的客户端和服务端对象，就可以理解为图中的客户端存根(Client Stub)、服务端存根(Server Stub)。



RPC内部功能模块

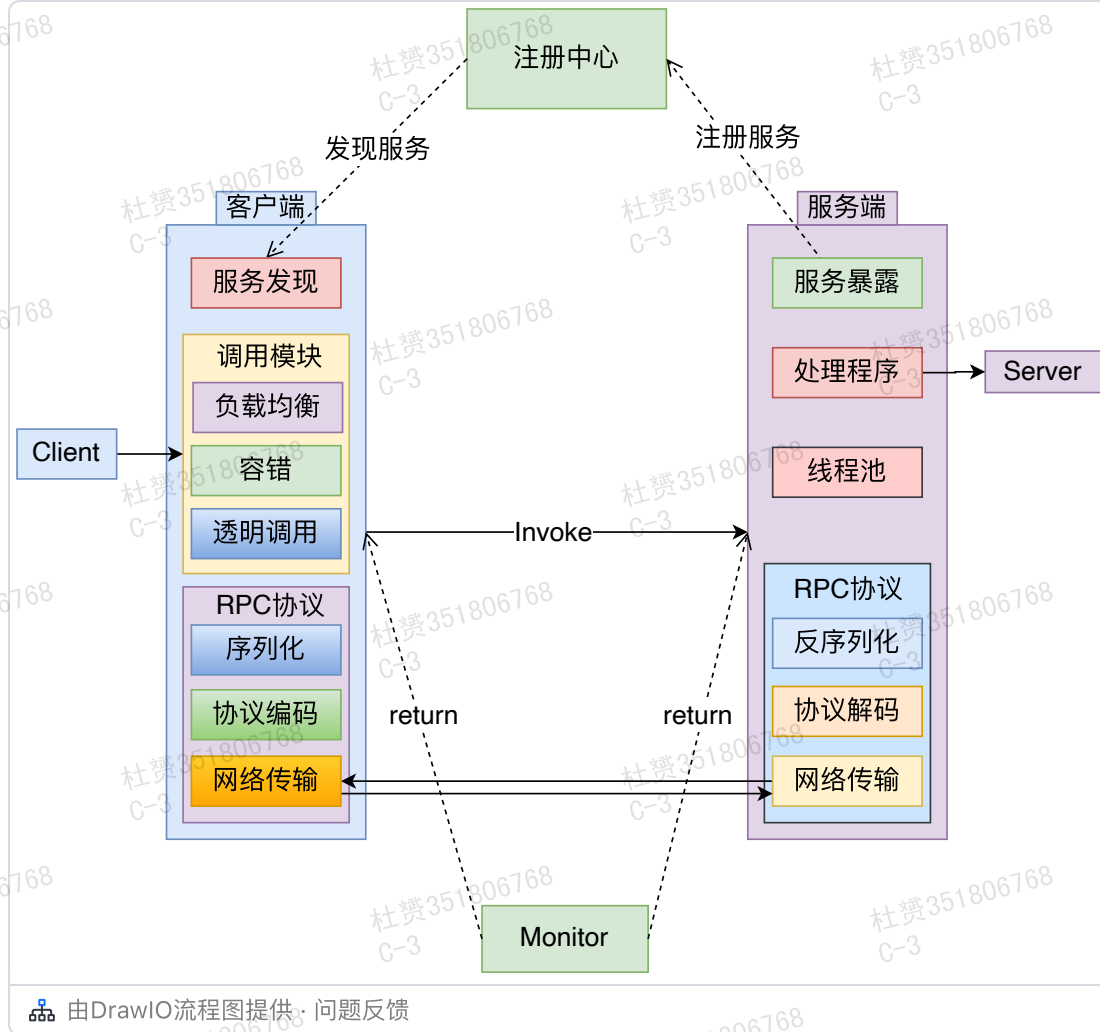
(1) **服务发现** (目标调用服务 -> ip地址列表): 通过注册中心管理所有的服务、记录它们所有已部署机器的ip地址, 不同服务之间可以通过注册中心互相发现, 并获取对方服务的ip地址列表。

(2) **负载均衡** (ip地址列表 -> 具体调用ip): 客户端将上述得到的appkey对应ip地址列表的映射保存在一个本地缓存里, 并通过负载均衡策略确定每次调用实际目标服务器ip。

(3) **代理调用**: RPC框架使用被动态代理后的客户端和服务端对象进行调用和方法执行, 并在请求信息中携带目标调用接口的接口名、方法名, 供服务端定位并执行目标方法。

(4) **序列化和反序列化**: 将客户端请求或服务端回传的对象数据与二进制数据互相转换, 使用字节流进行下一层的网络传输, 这里就能实现跨语言的功能 (序列化和反序列化的方式有很多种)。

(5) **网络通信**: 使用tcp或http协议来进行网络通信, 实现请求的发送和接收 (一般是tcp, 效率高), 这里可以使用几种不同的IO模型 (BIO、NIO) 作为实现Socket逻辑的核心模型 (一般NIO)。



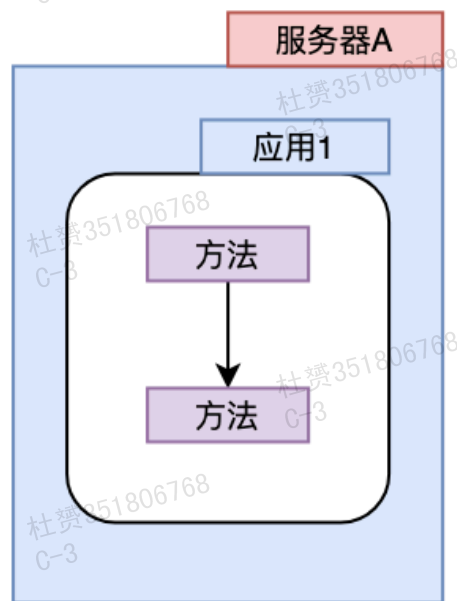
RPC介绍二

一、简介

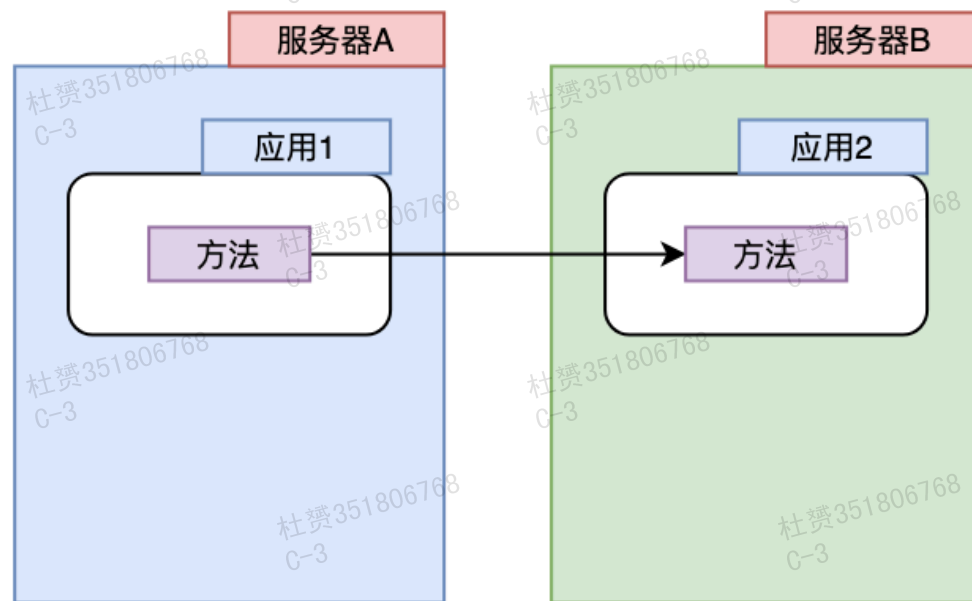
1.1、什么是RPC?

RPC (Remote Procedure Call) 即**远程过程调用**，其对应的是我们的【本地过程调用LPC】。

本地调用



远程调用

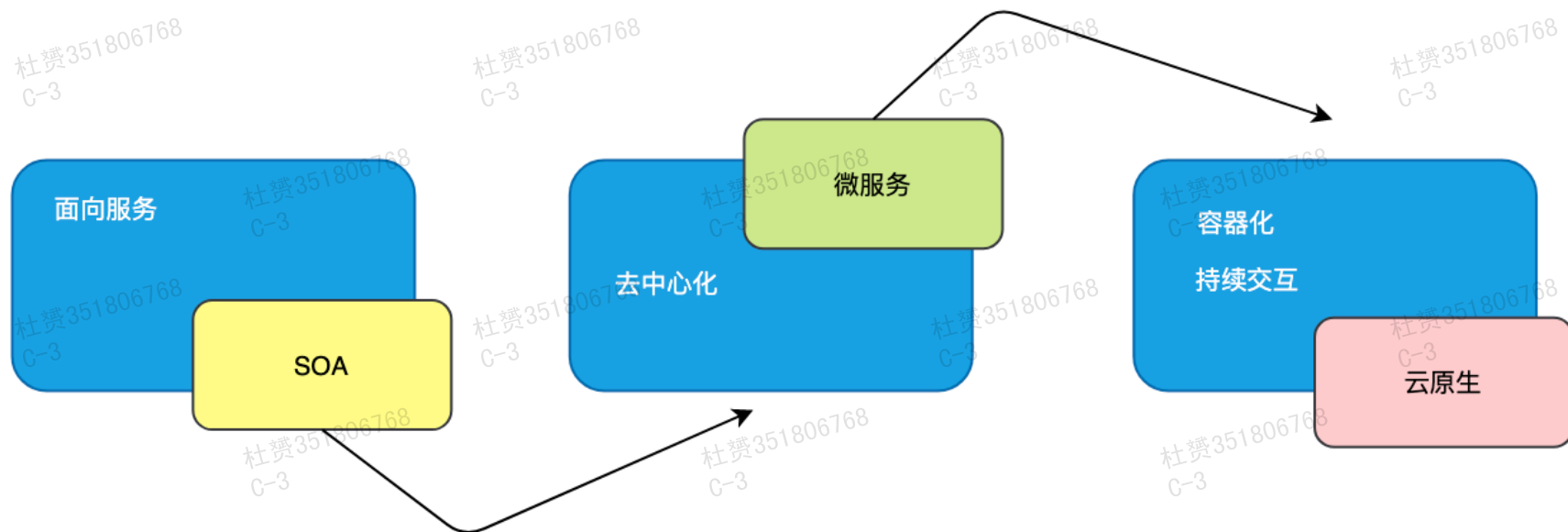


1.2、什么是“远程”？

远程其实指的就是需要通过网络通信，RPC 是一种技术概念，该概念允许运行于一台计算机的程序调用另一台计算机的子程序（两个或多个应用程序都分布在不同的服务器上），而开发人员无需额外地为这个交互编程（开发者无需额外地为这个调用过程编写网络通信相关代码细节）。

所以RPC简单理解就是调用远程机器上的函数或方法，它就像是调用本地函数或方法一样。

1.3、为什么需要RPC？



简单讲就是应用开发到一定的阶段的强烈需求驱动的，大量的服务和流量的业务，需要对单体架构和垂直架构进行升级，使用微服务+分布式架构进行开发，而RPC就是在分布式架构中更好的支持业务，主要用于服务器集群间的通信，特点是序列化/反序列化更快，传输更快。RPC是构建分布式的基石。

1.3.1、跟我们常用的HTTP对比

	RPC	HTTP
性能	主要差别在序列化和反序列化 RPC通常基于二进制传输	HTTP一般是json格式 序列化更消耗性能

传输协议	RPC大多基于TCP，也可以基于HTTP（长连接）	HTTP只能是HTTP
传输效率	可自定义TCP协议，报文相对较小 或者使用HTTP2协议，也可以很好的减少报文的体积，提高传输效率	HTTP报文有很多无用信息（头部信息）

1.3.2、RPC要解决的问题是

如果没有统一的RPC框架，各个团队的服务提供方就需要各自实现一套序列化、反序列化、网络框架、连接池、收发线程、超时处理、状态机等“业务之外”的重复技术劳动，造成整体的低效。

1.4、RPC优势不足

优势

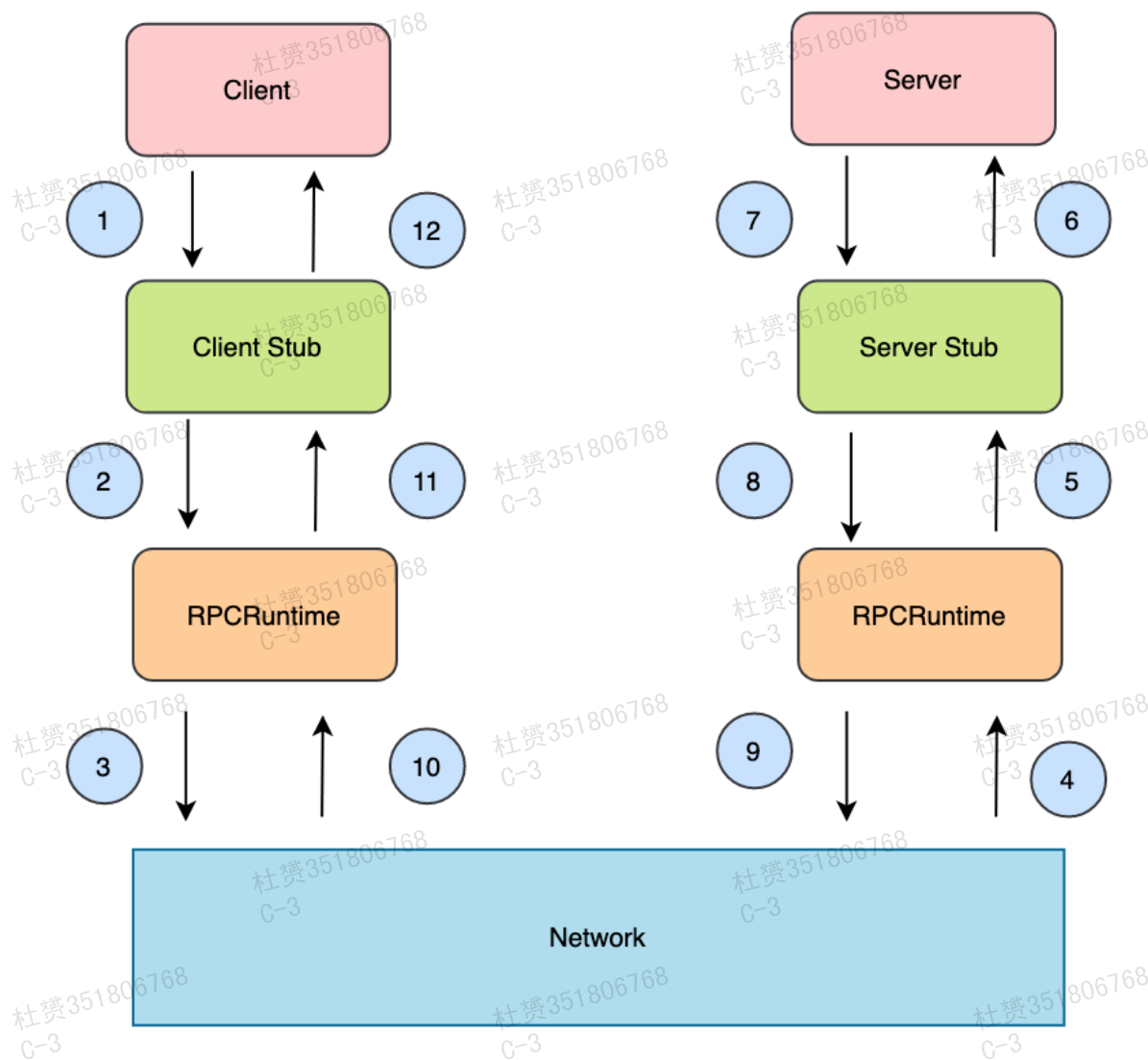
1. 屏蔽远程调用和本地调用的区别，让我们感觉像是调用本地方法一样；
2. 隐藏底层网络的复杂性，让我们更专注于业务逻辑；
3. 通信传输更快，序列化/反序列化更快；
4. 提升系统可扩展性；
5. 提升系统可维护性和持续交付能力；
6. 实现系统高可用；

RPC的不足（也是分布式架构的痛点）

1. 通信延迟的问题
2. 地址空间被隔离
3. 局部故障
4. 并发问题

二、RPC调用流程介绍

RPC调用的基本流程



RPC一般需要经历4个大步骤

1、建立通信

A机器想要调用B机器，首先得在客户端和服务端之间建立连接。

2、服务寻址

要解决寻址的问题，A服务器上如何连接到B服务器（如主机或IP地址）以及特定的端口，方法的名称是什么。

3、网络传输

1) 序列化

当A服务器上的应用发起一个RPC调用时，调用方法和参数数据都需要先进行序列化（转换为二进制）。

2) 反序列化

当B服务器接收到A服务器的请求之后，又需要对接收到的参数等信息进行反序列化操作（将二进制转换为数据，方便处理）。

4、服务调用

B服务器进行本地调用（通过代理Proxy）之后得到了返回值，此时还需要再把返回值发送回A服务器，同样也需要经过序列化操作，然后再经过网络传输将二进制数据发送回A服务器。

通常，一次完整的RPC调用需要经历如上4个步骤。

详细步骤如下：

(1) 服务消费者以本地调用方式（即以接口的方式）调用服务，它会将需要调用的方法、参数类型、参数传递给服务消费方的本地存根。(2) 服务消费方的本地存根收到调用后，负责将方法、参数等数据组装成能够进行网络传输的消息体（将消息体对象序列化为二进制数据），并将该消息体传输给 RPC 通信者。



(3) Consumer 端的 RPC 通信者通过 sockets 将消息发送到 Provider 端，由 Provider 端的 RPC 通信者接收。Provider 端将收到的消息传递给服务提供方的本地存根。

(4) 服务提供方的本地存根收到消息后将消息对象反序列化。

(5) 服务提供方的本地存根根据反序列化的结果解析出服务调用的方法、参数类型、参数等信息，并调用服务提供方的服务。

(6) 服务提供方执行对应的方法后，将执行结果返回给服务提供方的本地存根。(7) 服务提供方的本地存根将返回结果序列化，并且打包成可传输的消息体，传递给 Provider 端的 RPC 通信者。

(8) Provider 端的 RPC 通信者通过 sockets 将消息发送到 Consumer 端，由 Consumer 端的 RPC 通信者接收。Consumer 端将收到的消息传递给服务消费方的本地存根。

(9) 服务消费方的本地存根收到消息后将消息对象反序列化。反序列化出来的是方法执行的结果，并将结果传递给服务消费者。

(10) 服务消费者得到最终执行结果。

三、RPC核心技术

1. 通信协议

2. 序列化

简单讲，通信协议描述的就是请求头，而序列化关注的就是请求体。

3.1、通信协议

一般都是基于TCP实现自定义协议（如Thrift），或者基于HTTP/2实现（如gRPC），当然也有很多已经实现的协议，如下：

1. CORBA (Common Object Request Broker Architecture)：古老而复杂，支持面向对象的通信协议

2. Web Service (SOA SOAP RDDI WSDL)：基于http+xml的标准化Web API；

3. RestFul (Reprensentational State Transfer)：http+json；

4. RMI (Remote Method Invocation)：Java内部的分布式通信协议；

5. JMS (Java Message Service) : JavaEE的消息框架标准, 很多MQ支持;

6. Hessian

7. HTTP/2

8. Protobuf(Protocol Buffers, Protobuf是一种更加灵活、高效的数据格式, 与XML、JSON类似, 但在一些高性能且对响应速度有要求的数据传输场景中非常适用, 它会将字符串进行序列化后再进行传输, 即二进制数据传输, 详见< <https://github.com/protocolbuffers/protobuf> >)

9. Thrift: TCP+二进制

3.2、序列化

序列化和反序列化是一种数据转化的技术, 通常使用序列化技术指代即可。

序列化技术的出现就是因为数据有存储和传输的需求, 所以从数据的用途来看, 序列化就是为了将数据按照规定的格式进行重组, 在保持原有的数据语义不变的情况下, 达到存储或者传输的目的。反序列化则是为了将序列化后的数据重新还原成具有语义的数据, 以达到重新使用或者复用原有数据的目的。

序列化技术的实现方案, 根据序列化后的数据格式可以分为以下两种:

1. 文本格式的序列化实现方案, 如JSON、XML、Avro等

2. 二进制格式的序列化实现方案, 如Hessian、ProtoBuf

业界大多RPC框架还是基于二进制格式的序列化实现方案来做的, 如Thrift、gRPC等

RPC实现多台机器的调用通信, 本质上都是借助底层TCP/IP协议, 通过序列化和反序列化实现的。

四、简易RPC实现

简易RPC实现的代码: <http://www.broadview.com.cn/book/6459>

五、市面上的RPC框架

相关框架

1. sofa-rpc (蚂蚁金服)
2. RMI (Sub/Oracle)
3. Finagle (Twitter)
4. bRPC (百度)
5. tRPC (腾讯)
6. sRPC (搜狗, 基于WF)
7. RPCX
8. KiteX (头条)
9. Motan (新浪微博)
10. MTThrift (美团)
11. Dubbo (阿里)
12. Thrift (Meta -> Apache)
13. gPRC (Google)
14. Spring Cloud

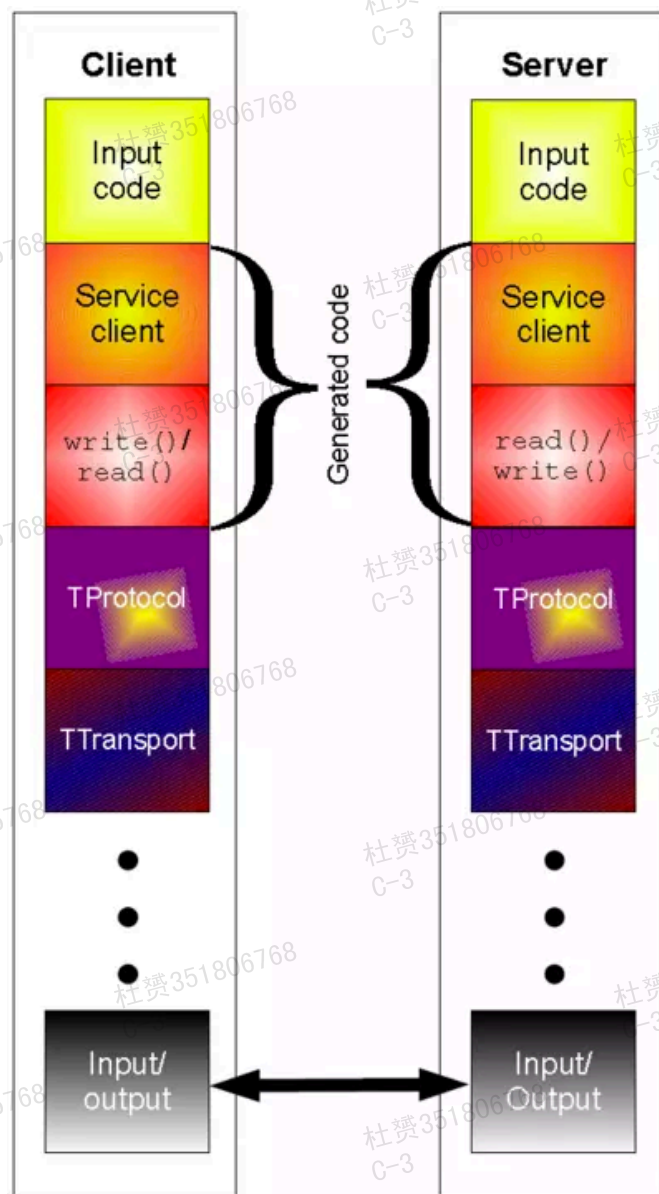
从业界来看, 推荐后四种RPC框架

六、Thrift介绍

Thrift是一个轻量级、跨语言的远程服务调用框架, 最初由Facebook开发, 后面进入Apache开源项目。它通过自身的IDL中间语言, 并借助代码生成引擎生成各种主流语言的RPC服务端/客户端模板代码。

Thrift支持多种不同的编程语言, 包括C++、Java、Python、PHP、Ruby、NodeJS等, 本系列主要讲述基于Java语言的Thrift的配置方式和具体使用。

6.1、设计架构



Thrift软件栈分层从下向上分别为：

- **传输层(Transport Layer)：**传输层负责直接从网络中读取和写入数据，它定义了具体的网络传输协议；比如说TCP/IP传输等。
- **协议层(Protocol Layer)：**协议层定义了数据传输格式，负责网络传输数据的序列化和反序列化；比如说JSON、XML、二进制数据等。



- **处理层(Processor Layer)**: 处理层是由具体的IDL (接口描述语言) 生成的, 封装了具体的底层网络传输和序列化方式, 并委托给用户实现的Handler进行处理。
- **服务层(Server Layer)**: 整合上述组件, 提供具体的网络线程/IO服务模型, 形成最终的服务。

6.2、Thrift优缺点

1. 支持非常多的语言绑定。
2. 通过CLI把thrift文件生成目标代码, 简单易用。
3. 数据结构与传输表现的分离, 支持多种消息格式。
4. 包含完整的客户端/服务端堆栈, 可快速实现RPC。
5. 支持同步和异步通信。
6. 稳定/广泛使用
7. 和protobuf(谷歌的一种灵活高效的独立于语言平台的结构化数据表示方法)一样不支持动态特性

6.3、Thrift的数据类型

Thrift 脚本可定义的数据类型包括以下几种类型:

1. **基本类型**:
 - **bool**: 布尔值
 - **byte**: 8位有符号整数
 - **i16**: 16位有符号整数
 - **i32**: 32位有符号整数
 - **i64**: 64位有符号整数
 - **double**: 64位浮点数



- **string**: UTF-8编码的字符串

- **binary**: 二进制串

2. 结构体类型:

- **struct**: 定义的结构体对象

3. 容器类型:

- **list**: 有序元素列表
- **set**: 无序无重复元素集合
- **map**: 有序的key/value集合

4. 异常类型:

- **exception**: 异常类型

5. 服务类型:

- **service**: 具体对应服务的类

6.4、Thrift的协议

Thrift可以让用户选择客户端与服务端之间传输通信协议的类别，在传输协议上总体划分为文本(text)和二进制(binary)传输协议。为节约带宽，提高传输效率，一般情况下使用二进制类型的传输协议为多数，有时还会使用基于文本类型的协议，这需要根据项目/产品中的实际需求。常用协议有以下几种:

- TBinaryProtocol: 二进制编码格式进行数据传输
- TCompactProtocol: 高效率的、密集的二进制编码格式进行数据传输
- TJSONProtocol: 使用JSON文本的数据编码协议进行数据传输
- TSimpleJSONProtocol: 只提供JSON只写的协议，适用于通过脚本语言解析

6.5、Thrift的传输层

常用的传输层有以下几种：

- TSocket：使用**阻塞式I/O**进行传输，是最常见的模式
- TNonblockingTransport：使用**非阻塞方式**，用于构建**异步客户端**
- TFramedTransport：使用**非阻塞方式**，按块的大小进行传输，类似于Java中的NIO

6.6、Thrift的服务端类型

- TSimpleServer：**单线程**服务器端，使用标准的**阻塞式I/O**
- TThreadPoolServer：**多线程**服务器端，使用标准的**阻塞式I/O**
- TNonblockingServer：**单线程**服务器端，使用**非阻塞式I/O**
- THsHaServer：**半同步半异步**服务器端，基于**非阻塞式IO**读写和**多线程**工作任务处理
- TThreadedSelectorServer：**多线程选择器**服务器端，对THsHaServer在**异步IO**模型上进行增强

6.7、Thrift入门示例

Node.js Tutorial: <https://thrift.apache.org/tutorial/nodejs.html>

6.8、公司内部MTThrift示例

- MTThrift快速使用: [快速入门](#)
- ThriftHub: <https://thrifthub.sankuai.com/compile>

代码块

JavaScript ▾



定义IDL文件

代码块

JavaScript ▾



```
1 namespace java com.sankuai.meituan.meishi.merchant.photo.c.thrift
2
3 struct THeadPic {
4     i32 picId;
5     string title;
6     string thumbUrl;
7     string url;
8     double price;
9     string uploader;
10    i64 uploadTime;
11    string videoUrl; // 不空表示有主图视频
12    i32 videoSize; // 视频大小, 单位字节, 如果视频存在的情况下这个字段才有意义
13    i32 videoDuration; // 视频播放时长, 单位秒, 如果视频存在的情况下这个字段才有意义
14    string originalUrl; // 原始url
15    i64 videoId;
16 }
17
18 struct TOfficialAlbum {
19     i32 id;
20     i32 shopId;
21     string name;
22     i32 albumType;
23     i32 picType;
24     i32 picId;
25     string picPath;
```

```
26     i32 picCount;
27     i64 addTime;
28     i64 updateTime;
29 }
30
31 struct TOfficialPic {
32     string title; // 图片标题
33     i32 picType; // 图片类型 1-环境 2-菜品 11-价目表 10-其他
34     double price; // 菜品图片中菜品的价格, 如不是菜品图片该字段值为0
35     string smallPicUrl; // 小图URL
36     string middlePicUrl; // 中图URL
37     string bigPicUrl; // 大图URL
38     string uploader; // 上传者
39     i64 uploadTime; // 上传时间
40     string originalUrl; // 原始图片
41     i32 picId;
42     bool isHeadPic;
43
44 }
45
46 struct TOfficialVideo {
47     /**
48      * 视频id
49      */
50     1:i64 id;
51     /**
52      * 视频url
53      */
54     2:string url;
55     /**
56      * 帧图片
57      */
58     3:string frameUrl;
```



```
92
93 enum THeadPicModeEnum {
94     /**
95      * 大图模式
96      */
97     BIG_MODE = 1,
98     /**
99      * 轮播模式
100     */
101     PLAY_MODE = 2,
102     /**
103      * 小图模式
104     */
105     SINGLE_MODE = 3
106 }
107
108 service TPhotoService {
109     list<THeadPic> getHeadPicByShopId(1:i32 shopId)
110
111     list<THeadPic> getHeadPicByPoiId(1:i32 poiId) //通过美团poiId获取头图列表
112
113     list<TOfficialAlbum> getOfficialAlbums(1:i32 shopId)
114
115     list<TOfficialAlbum> getPanoramicAlbums(1:i32 shopId)
116
117     list<TOfficialPic> getOfficialPics(1:i32 shopId,2:i32 picType)
118
119     i32 getOfficialPicsCountByShopId(1:i32 shopId)
120
121     list<TOfficialPic> getOfficialPicsByPoiId(1:i32 poiId,2:i32 picType) //通过美团poiId相册图片列表
122
123     list<TOfficialPic> getOfficialPicsByShopId(1:i32 shopId,2:string dpId)
124
```

map<i32, i32> getHeadPicModeByShopIds(1:list<i32> shopIds)

/**
 * 美团侧，播放模式，批量
 * add by gf 171208
 */

map<i32,i32> getHeadPicModeByPoiIds(1:list<i32> poiIds)

/**
 * 美团侧，官方视频库获取接口
 * add by gf 171208
 */

list<TOfficialVideo> getOfficialVideosByPoiId(1:i32 poiId)

/**
 * 点评侧，官方视频库获取接口
 */

list<TOfficialVideo> getOfficialVideosByShopId(1:i32 shopId)

/**
 * 点评侧，官方视频数量获取接口
 */

i32 getOfficialVideosCountByShopId(1:i32 shopId)

/**
 * 美团侧，头图，播放模式
 */

HeadPicPage getHeadPicPageByPoiId(1:i32 poiId);

/**
 * 批量获得主图视频
 */

map<i32,TOfficialVideo> getOfficialVideoMapByShopIds(1:list<i32> shopIds)

```
158 /**
159  * 获取主图的在ugc的id
160 **/
161 list<i32> getHeadPicUgcIds(1:i32 shopId);
162
163 /**
164  * 美团侧，菜品、环境、价目表、其他以及新增的菜单相册
165  * //todo 判断几个相册的返回顺序
166  * add by llr 180728
167 **/
168 list<TOfficialPic> getOfficialPicsByPoiIdV2(1:i32 poiId,2:i32 picType) //通过美团poiId相册图片列表
169
170 /**
171  * 美团侧，获取门店下的拍菜单相册
172  * add by llr 180728
173 **/
174 list<TOfficialPic> getMenuPicsByPoiId(1:i32 poiId) //通过美团poiId相册图片列表
175
176 /**
177  * 美团侧，判断拍菜单相册是否有图片
178  * add by llr 180728
179 **/
180 bool hasMenuPicsByPoiId(1:i32 poiId) //通过美团poiId相册图片列表
181
182 }
```

IDL转为NodeJS

代码块

JavaScript

thrift --gen js:node xxx.thrift

也可以通过公司的ThriftHub转换后，直接使用npm包

调用

代码块

JavaScript

```
1 const { BoardService } = require("@thrift-api/boardapi");
2
3 const boardService = new BoardService({
4   appkey: 'com.sankuai.web.meishife.mobilenext',
5   thriftOptions: {
6     remoteAppKey: "com.sankuai.web.meishiop.boardapi",
7     serviceName: "com.sankuai.meituan.boardapi.rpc.hotboard.BoardService",
8     ports: [9006],
9     timeout: 6000,
10    enableAuth: true,
11    unified: true
12    // mock: true, // 本地mock数据开关
13  }
14 });
15
16 boardService
17   .getAreaByLngLat({
18     request: params,
19   })
20   .catch(() => {
21     return 500;
22   })
23   .then(res => {
24     return res;
```

6.9、扩展：按照调用方式来看，RPC有四种模式：

1. RR (Request-Response) 模式，又叫请求响应模式，指每个调用都要有具体的返回结果信息。
2. Oneway模式，又叫单向调用模式，调用即返回，没有响应的信息。
3. Future模式，又叫异步模式，返回拿到一个Future对象，然后执行完获取到返回结果信息。
4. Callback模式，又叫回调模式，处理完请求以后，将处理结果信息作为参数传递给回调函数进行处理。

这四种调用模式中，RR和Oneway最常见。从本质上看，RPC一般对于客户端来说是一种同步的远程服务调用技术。

七、参考

1. RPC框架：从原理到选型，一文带你搞懂RPC: <https://mp.weixin.qq.com/s/Il4nUVB28KpyTMS93xAckQ>
2. RPC编程（一）：初步了解RPC: <https://mp.weixin.qq.com/s/dmqgbMdwZNsjCiWvFtcSbg>
3. 100行代码透彻解析RPC原理: <https://mp.weixin.qq.com/s/INiC1pjsaxcaROQGyiyauw>
4. Thrift系列详解: <https://juejin.cn/post/6844903622380093447>
5. 《深入理解RPC: 框架原理与实现》华钟明
6. [RPC实战与核心原理](#) 何小锋

MTthrift使用

引入mtthrift依赖

(服务端、客户端服务都需要引入依赖，其实已经建好的mdp项目不需要手动引mtthrift依赖，已经在mdp的parent依赖中引入了)

代码块

```
1 // mdp项目父级依赖中已经引入了 https://km.sankuai.com/page/1303120011
2 <parent>
3     <groupId>com.meituan.mdp</groupId>
4     <artifactId>mdp-parent</artifactId>
5     <version>1.7.1.1-GROCERY-RC5</version>
6 </parent>
7
8 // 实际引用到的mtthrift包
9 <dependency>
10     <groupId>com.meituan.service.mobile</groupId>
11     <artifactId>mtthrift</artifactId>
12     <version>2.3.2</version>
13     <scope>compile</scope>
14 </dependency>
```

服务端

1、client包中定义接口

thrift接口的定义其实有两种方式，IDL文件生成（很旧，尽量不使用），和使用thrift注解定义（推荐用这种方式）。

(1) IDL文件生成：先在指定位置按需求创建接口定义文件xxx.thrift，里面描述要定义的类(struct)、接口方法(service)等。

使用maven的Dolphin插件 plugins->genthrift->genthrift:run 根据thrift描述文件自动生成java代码

(2) Thrift注解方式定义接口：@ThriftStruct 标识入参、返回值类定义、@ThriftService 标识thrift接口定义

(其余这里的@InterfaceDoc、@MethodDoc、@TypeDoc 都是接口SOA文档生成注解，与thrift接口定义无关)

代码块

```
1 // 一、IDL方式定义接口
2 // 1、入参、返回值类定义 client层 resources文件夹下 thrift包 创建Hello.thrift文件 (struct->定义类)
3 namespace java com.sankuai.grocerypoi.cquery.client.hello
4
5 struct HelloResponse{
6     1: i32 code;
7     2: string msg;
8 }
9
10 // 2、接口定义 HelloService.thrift (service->定义服务接口)
11 namespace java com.sankuai.grocerypoi.cquery.client
12
13 include "Hello.thrift" // 引用thrift文件
14 include "Response.thrift"
15
16 typedef Hello.HelloResponse HelloResponse // 引用thrift文件下的类
17 typedef Response.VirtualPoiTResponse VirtualPoiTResponse
18
19 service HelloService {
20     HelloResponse gethello(1: string id); // 接口中的方法, 1: 后面标识参数 注意必须有空格
21 }
22
23 // 3、插件依赖
24 <plugin>
25     <groupId>com.sankuai.octo</groupId>
26     <artifactId>genthrift-maven-plugin</artifactId>
27     <version>1.0.0-SNAPSHOT</version>
28     <configuration>
29         <appkey>com.sankuai.grocerypoi.bmquery</appkey>
30         <idlDir>src/main/resources/thrift</idlDir>
31     </configuration>
32 </plugin>
```

```
33
34
35
36 // 二、注解方式定义接口
37 // 1、入参、返回值类定义 @ThriftStruct注解定义类 (这里省略字段的get、set、toString等方法)
38 @TypeDoc(
39     description = "mtthrift调用示例服务"
40 )
41 @ThriftStruct
42 public class HelloResponse {
43
44     @FieldDoc(
45         description = "返回值code"
46     )
47     private Integer code;
48
49     @FieldDoc(
50         description = "调用信息"
51     )
52     private String msg;
53 }
54
55 // 2、接口定义 @ThriftService注解定义接口, 此service下的所有方法均为thrift方法
56 @InterfaceDoc(
57     name = "HelloService",
58     description = "mtthrift调用示例服务",
59     displayName = "mtthrift调用示例服务",
60     type = "octo.thrift.annotation",
61     scenarios = "mtthrift调用示例服务"
62 )
63 @ThriftService
64 public interface HelloService {
65
```

```

66     @MethodDoc(
67         displayName = "mtthrift调用示例接口",
68         description = "mtthrift调用示例接口",
69         parameters = {
70             @ParamDoc(name = "id", description = "入参id", rule = "不能为空"),
71         },
72         extensions = {
73             @ExtensionDoc(name = "SECURITY_PRIVILEGE", content = "登录鉴权")
74         }
75     )
76     HelloResponse gethello(String id);
77 }

```

2、server层中实现上述接口

代码块

```

1 package com.sankuai.grocerypoi.cquery.hello;
2
3 import com.sankuai.grocerypoi.cquery.client.HelloService;
4 import com.sankuai.grocerypoi.cquery.client.hello.Hello1;
5 import com.sankuai.grocerypoi.cquery.client.response.BatchGetVirtualPoiIdByStoreIdResponse;
6 import com.sankuai.grocerypoi.cquery.client.response.VirtualPoiResponse;
7 import org.apache.thrift.TException;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 public class HelloServiceImpl implements HelloService { //实现thrift接口逻辑
12
13     @Override
14     public HelloResponse gethello(String id) throws TException {
15         HelloResponse response = new HelloResponse();

```

```

16 response.setCode(0);
17 response.setMessage("hello "+id);
18 return response;
19 }
20 }

```

3、将client打成jar包，供客户端引用

初次执行maven deploy 将jar包上传到pixel（美团的maven仓库）之前需要有一些配置，具体可参考其它文档。

（注意：非正式版本的包以SNAPSHOT结尾，才可多次修改上传，正式版打包后无法修改和回退）

4、部署服务端服务

客户端

1、引用服务端接口jar包

2、编写实际调用代码，先通过@MdpThriftClient(服务端appkey) 注解，指定要调用的服务，然后像调用本地方法一样编写调用远程方法代码

（实际调用中需要先确保服务端添加了客户端的服务鉴权）

父pom中管理版本，子pom中再实际引用

```

1 <dependency>
2   <groupId>com.sankuai.grocery.product</groupId>
3   <artifactId>grocery-product-schedule-mapi-api</artifactId>
4   <version>0.0.56</version>
5 </dependency>

```

客户端实际调用代码（线下调用需指定调用到服务端所部署泳道）

```

1 // 1、指定要调用的服务，helloService会被自动注入HelloService类的bean对象
2 @MdpThriftClient(

```

```
remoteAppKey = "com.sankuai.grocerypoi.cquery", timeout = 3000
4    )
5    private HelloService helloService;
6
7    // 2、实际调用代码
8    @Test
9    public void gethelloTest() {
10        //Tracer.setSwimlane("20313-xxxxx"); 线下环境：指定调用泳道为服务端的部署泳道
11        HelloResponse response = helloService.gethello("1000000");
12        System.out.println(JsonUtils.toJsonWithoutException(response));
13    }
```

一、底层网络通信架构原理（核心基础）

1. Reactor网络模型深度解析

主从Reactor架构设计：

MTthrift采用经典的主从Reactor模式，这是一种高性能网络服务器的标准架构。主Reactor负责接收新连接，从Reactor负责处理已建立连接的IO事件。这种设计实现了连接建立与数据处理的完全分离，避免了单点瓶颈。

事件驱动机制：

整个网络层基于事件驱动模型构建。当网络事件发生时，如连接建立、数据到达、连接断开等，会触发相应的事件处理器。这种机制使得单个线程能够同时处理数千个并发连接，大幅提升了系统的并发处理能力。

IO多路复用实现：

底层使用epoll（Linux）或kqueue（macOS）等高效的IO多路复用技术。这些技术允许一个线程监控多个文件描述符的状态变化，只有当真正有数据需要处理时才会唤醒处理线程，避免了传统多线程模型中的上下文切换开销。

2. 连接管理与复用机制

连接池的分层设计：

MTthrift的连接池采用多层架构，包括核心连接池、热备连接池和冷备连接池。核心连接池维护常用的活跃连接，热备连接池应对突发流量，冷备连接池则在极端情况下提供兜底保障。

连接健康检测算法：

采用自适应的健康检测机制，根据历史调用成功率、响应时间等指标动态调整检测频率。对于高质量连接降低检测频率减少资源消耗，对于不稳定连接增加检测频率确保及时发现问题。

连接复用策略：

NettyIO模式支持真正的连接复用，一个TCP连接可以同时承载多个RPC请求。这通过请求ID关联机制实现，每个请求分配唯一ID，响应通过ID匹配回对应的请求上下文，从而实现请求与响应的正确关联。

二、动态代理与服务调用机制（调用核心）

3. 多层代理架构设计

接口代理层：

MTthrift通过JDK动态代理或CGLIB字节码增强技术，为每个服务接口生成代理对象。这个代理对象拦截所有方法调用，将本地方法调用转换为远程网络调用。代理层负责参数封装、调用路由、结果解包等核心功能。

调用链路构建：

当客户端调用远程方法时，代理对象构建完整的调用链路。这包括调用上下文创建、请求序列化、网络传输、响应反序列化、异常处理等步骤。整个过程对业务代码完全透明。

异步调用机制：

MTthrift支持同步和异步两种调用模式。异步调用通过Future或CompletableFuture实现，允许调用方在等待响应期间处理其他业务逻辑。异步调用的核心是回调机制和事件循环，确保响应能够正确路由到原始调用方。

4. 方法调用拦截与增强

AOP拦截器链：

MTthrift内置了完整的AOP拦截器链，支持在方法调用前后插入自定义逻辑。这些拦截器包括参数校验、权限验证、性能监控、异常处理等。拦截器按照优先级有序执行，形成完整的处理链路。

上下文传播机制：

支持跨服务的上下文传播，如用户身份、请求追踪ID、业务标识等信息能够在服务调用链中自动传递。这通过ThreadLocal变量和协议头部字段实现，确保上下文信息的完整性和一致性。

三、序列化与协议栈设计原理（性能关键）

5. Thrift序列化深度优化

二进制紧凑编码：

MTthrift使用改进的Thrift二进制协议，采用变长编码技术最小化数据大小。对于小整数使用更少的字节表示，对于字符串和集合类型使用长度前缀编码，整体数据大小比JSON减少60%以上。

零拷贝序列化：

在序列化过程中直接操作ByteBuffer，避免不必要的内存拷贝操作。数据从应用对象直接序列化到网络缓冲区，减少了中间环节的内存分配和拷贝，显著提升序列化性能。

类型系统优化：

MTthrift对Thrift的类型系统进行了扩展，支持更丰富的数据类型，如泛型、枚举、嵌套结构等。同时优化了类型元数据的存储，减少了序列化数据的冗余信息。

6. 协议栈分层设计

传输层协议：

基于TCP协议构建私有应用层协议，包含魔数、版本号、消息类型、序列号、头部长度、消息体长度等字段。这种设计既保证了协议的可扩展性，又确保了版本兼容性。

消息分帧机制：

采用长度前缀的分帧策略，解决TCP粘包拆包问题。每个消息包含固定长度的头部，头部包含消息体长度信息，接收方根据长度信息准确分割消息边界。

压缩与加密支持：

协议层支持可选的数据压缩和加密功能。压缩算法包括gzip、snappy等，可根据数据特征自动选择最优压缩算法。加密支持AES、RSA等算法，确保敏感数据的传输安全。

四、服务发现与负载均衡原理（分布式核心）

7. 服务注册发现机制

分布式服务注册表：

MTthrift集成MNS（美团命名服务）作为服务注册中心，构建了高可用的分布式服务注册表。服务实例启动时自动注册到注册中心，包含服务名、版本、地址、权重、健康状态等元数据。

服务发现缓存策略：

客户端本地维护服务实例缓存，定期从注册中心同步最新的服务列表。采用增量更新机制，只传输变更的服务信息，减少网络开销。本地缓存支持多级存储，包括内存缓存和磁盘缓存，确保在网络分区时的服务可用性。

健康检查与故障检测：

实现了多维度的健康检查机制，包括心跳检测、业务接口探测、系统资源监控等。采用滑动窗口算法统计服务实例的健康指标，根据历史数据预测服务健康趋势，提前发现潜在问题。

8. 负载均衡算法实现

自适应负载均衡：

MTthrift实现了多种负载均衡算法，包括随机、轮询、加权轮询、最少活跃调用等。特别是最少活跃调用算法，能够根据服务实例的实时负载情况动态分配请求，实现真正的负载均衡。

一致性哈希优化：

对于有状态服务，使用一致性哈希算法确保相同键值的请求路由到同一服务实例。通过虚拟节点技术解决数据倾斜问题，每个物理节点映射到多个虚拟节点，确保负载分布的均匀性。

粘性会话支持：

支持基于用户会话的粘性路由，确保同一用户的请求始终路由到同一服务实例。这对于有会话状态的应用非常重要，避免了分布式会话同步的复杂性。

五、容错与可靠性保障机制（高可用核心）

9. 多层次超时控制

分层超时策略：

MTthrift实现了细粒度的超时控制，包括连接超时、读写超时、方法级超时、服务级超时等。不同层次的超时相互配合，形成完整的超时保护机制。超时参数支持动态配置，可根据业务需求实时调整。

自适应超时算法：

基于历史调用数据动态计算合理的超时时间。算法考虑平均响应时间、响应时间方差、网络抖动等因素，使用统计学方法确定最优超时阈值，既避免了过早超时，又防止了长时间等待。

10. 智能重试机制

指数退避重试：

采用指数退避算法控制重试间隔，避免重试风暴对下游服务造成冲击。重试间隔从初始值开始，每次重试后按指数递增，同时引入随机抖动避免多个客户端同时重试。

异常分类重试：

根据异常类型智能决定是否重试。对于网络超时、连接中断等临时性异常进行重试，对于业务异常、参数错误等永久性异常不进行重试，避免无效的重试操作。

11. 熔断降级保护

熔断器状态机：

实现了完整的熔断器状态机，包括关闭、打开、半开三种状态。熔断器根据失败率、响应时间等指标自动切换状态，在服务异常时快速熔断，避免雪崩效应。

降级策略引擎：

支持多种降级策略，包括返回默认值、调用本地缓存、切换备用服务等。降级策略可以通过配置文件或动态配置系统实时调整，确保在异常情况下仍能提供基本服务能力。

六、性能优化的核心技术原理（高性能基础）

12. 内存管理优化

对象池化技术：

MTthrift广泛使用对象池技术，包括连接池、缓冲区池、序列化对象池等。这些对象池减少了频繁的对象创建和销毁，降低了GC压力，提升了系统性能。对象池支持动态调整大小，根据系统负载自动扩缩容。

内存预分配策略：

在关键路径上预分配内存缓冲区，避免运行时的内存分配操作。特别是在序列化和网络IO过程中，使用预分配的DirectByteBuffer，减少堆内存使用，提升IO性能。

13. 批量处理优化

请求批量化：

支持将多个小请求合并为一个请求进行传输，减少网络往返次数。批量处理在序列化、网络传输、反序列化等环节都能带来显著的性能提升，特别适用于高并发小请求场景。

响应流水线处理：

在服务端实现流水线处理机制，将请求解析、业务处理、响应序列化等操作并行执行。这种设计充分利用了多核CPU的并行处理能力，提升了整体吞吐量。

七、分布式治理的实现机制（企业级特性）

14. 配置管理与动态更新

分层配置体系：

MTthrift实现了完整的分层配置体系，包括默认配置、环境配置、应用配置、服务配置、方法配置等。不同层次的配置按优先级生效，高优先级配置覆盖低优先级配置。

配置热更新机制：

集成Lion配置系统，支持配置的实时更新。当配置发生变更时，系统能够在不重启服务的情况下动态加载新配置，确保配置变更的即时生效。

15. 监控与可观测性

多维度指标采集：

MTthrift内置了完整的指标采集体系，包括调用量、响应时间、错误率、并发数等核心指标。指标采集采用异步方式，不影响正常的业务调用性能。

分布式链路追踪：

集成分布式链路追踪系统，能够追踪请求在整个服务调用链中的执行路径。每个请求分配全局唯一的TraceID，跨服务调用时自动传播，形成完整的调用链路图。

16. 服务治理自动化

智能路由决策：

基于服务实例的历史性能数据、当前负载状况、网络延迟等因素，智能选择最优的服务实例。路由决策算法持续学习和优化，不断提升路由效果。

自动扩缩容支持：

支持与容器编排系统集成，根据服务负载情况自动触发扩缩容操作。当服务压力增大时自动扩容增加服务实例，当压力减小时自动缩容释放资源。

MTthrift作为美团内部的核心RPC框架，其设计和实现体现了大规模分布式系统的最佳实践。从底层的网络通信到上层的服务治理，每个环节都经过精心设计和优化，形成了完整的企业级RPC解决方案。这些技术原理不仅适用于MTthrift本身，也为理解其他RPC框架提供了重要参考。

Dubbo

🔒 仅供内部使用，未经授权，切勿外传